DESIGN REPORT

This project demonstrates a working concurrent task dispatcher using Rust's own standard concurrency tools. System simulates how tasks arrive over time, will enter a queue, and are scheduled by a dispatcher which are now executed by a bounded worker pool. This project's main purpose is to focus on demonstrating the understanding of the concepts learned in class about concurrency, task scheduling, queue-based problems, worker pools, synchronization, and metrics.

1. **What threads OR major components exist in your design?**

The system contains four main components for this design: task generator thread, dispatcher thread, a pool of 8 worker threads and a shared metrics system.

This program acts as a simulation to mimic close to what might be seen in a real operating system. It mimics realistic scheduling behavior where a task generator creates incoming work/tasks over time and sends those tasks into the system. This also shows it in a way that it contains 8 'workers' and a total of 1000 tasks that will be generated automatically using the random fixed seed so that these experiments (Balanced or not) are given.

Which (the experiments) or scheduling policy will now show the workers executing these tasks with the tools given.

What is NEEDED in the system was a

- Task generator thread
- Dispatcher thread
- 8 workers thread (as required in the exalidraw document)
- Shared metrics collection
- And Queue-Based communication channels

```
37
38  #[derive(Clone, Debug)]
39  struct Task {
40      id: usize,
41      kind: TaskKind,
42      arrival_time: Instant,
43      duration_ms: u64,
44      cpu_cost: u32,
45  }
46
```

The given task includes two types which will behave differently.

CPU focused tasks are good for simulating heavier CPU usage while IO tasks simulate lighter CPU usage. These tasks will be used and given the concurrency tools from the Rust library will work between threads.

Mpsc channels

A main important tool is “mpsc” which can be found in the top of the code to implement it into the program.

use std:sync::{mpsc, Arc, Mutex}; // mpsc is for communication over channels…

Channels are much easier to use when it comes to transferring ownership of tasks safely between threads. It is preferable as well, and including threads differently is time-consuming and may lead to a possible problem in the future. Which is why mpsc is used instead. It allows the generator to send tasks to the dispatcher, and the dispatcher sends tasks to workers in the simulation, allowing it to avoid any unsafe access to task data.

2. **What data is shared, and how is it** protected**?**

This is actually protected by the Arc<Mutx<Metrics>>

Arc<Mutx<Metrics>>

These are shared between worker threads. Since this program will simulate many workers doing tasks and updating the progress at the same time, there should be something that prevents any problems. This is where the metrics structure is protected using a Metrex. Arc allows many multiple threads to safely share the ownership of the metrics.

There are other things necessary to make the programming run smoothly, such as…

- AtomicBool (for shutting down/signaling)
  This focuses on shutdown signaling where worker threads and the dispatcher repeatedly checks whether the program should stop running. This is necessary to avoid any infinite loops or lack of proper termination when the workers are done with the tasks.
- Worker Threads
  - For the system to progress, it uses a bounded worker pool (with the const WORKER_COUNT that contains 8) The purpose is to wait for tasks and

execute them, along with updating the progress when finished and continuing for more work if given.

- Recv_timeout()
    - Allows threads to check any shutdown conditions, preventing any hanging issues.

With this, it is possible for the generator to create task over time (task manager) which are sent into a central queue. The fn dispatcher THEN receives tasks and picks (which worker handles what), where the workers now proceed and program simulates concurrency.

After all the tasks are completed, the system will shut down smoothly without issues.

3. **Where did you use channels, and why?**

Channels are preferable for the benefits of safely transferring ownership of tasks between threads. This helps make the program transfer stuff smoothly without the worry of unsafe shared memory access.

4. **Where did you use shared state, and why?**
   The location of the shared state is the metrics and the shutdown signalling. These can be seen in Arc<Metex<Metrices>> and Arc<AtomicBool>
5. **What scheduling policy did you implement?**

FIFO and Optimized scheduling were implemented for the project.

FIFO: is where the policy distributes tasks evenly across workers in order (turnaround).

It also keeps the workload balanced and distributed to workers fairly.

While:

Optimized policy attempts to separate CPU heavy tasks from IO ones. This is shown through (Fifo 70% IO / 30% CPU) and Optimized, //Optimized 80%.

Tasks are always assigned to worker 0, (remaining tasks are distributed to the remaining)

Yet this scheduling policy shows a unbalanced and ‘unfair’ approach, acting like a ‘stressed’ situation.

Lastly, to properly demonstrate the difference between FIFO and optimized, it is important to run these where FIFO uses.

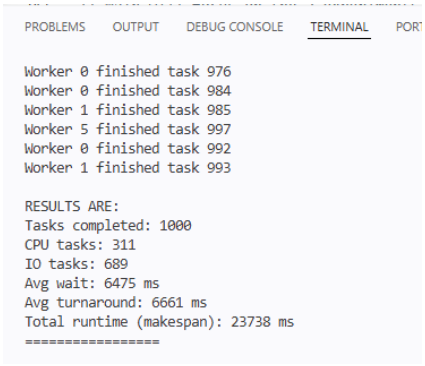
```
match policy{
    Policy::FIFO => {
        worker_senders[rr % WORKER_COUNT].send(task).unwrap();
        rr += 1;
    }

    Policy::Optimized => {
        // CPU tasks will now spread evenly go IO
        let target = match task.kind {
            TaskKind::CPU => 0,
            TaskKind::IO => (rr + 1) % WORKER_COUNT,
        };

        worker_senders[target].send(task).unwrap();
        rr += 1;
    }
}
```

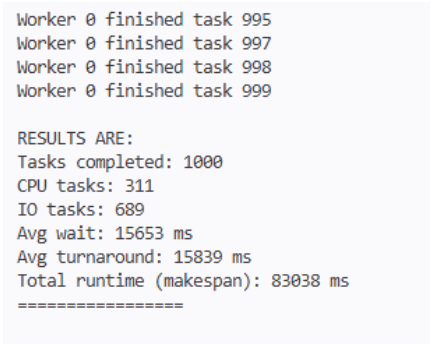
.

FIFO'S RESULTS | OPTIMIZED RESULTS



(FIFO RESULTS)

As shown here, the main difference is the avg wait, turnaround, and the required total_runtime of each policy.



(Optimized Results.)

6. **What behavior improved because of that policy?**

As shown from the results, the Optimized policy improves performance by isolating CPU heavy tasks and reducing interference between CPU and IO. This reduces any conflicts on these policies.

7. **What behavior became worse or more complicated because of that policy?**

However, this also meant Optimized is unbalanced compared to FIFO. While FIFO is balanced and distributed fairly, Optimized policy makes worker 0 receives all CPU tasks resulting overload.

OTHER QUESTIONS

8. **What concurrency bug or design mistake did you hit during development?**

I was having issues with the threading closing properly, as I got constant overflows or even certain crashes that wouldn't let me run the program.

9. **How did you fix it?**

The way I fixed it was using Arc<AtomicBool> which is part of Rust's library that allowed me also include with recv_timeout() allowing the threads to be checked over time to time whether the program should exit or not.

10. **Where could starvation or unfairness still happen?**

Out of the two policies, it can happen in Optimized policy because CPU tasks were constantly being given to worker 0. If worker 0 receives to much of this, it can overload and impact other workers/the system.